

CYBERNEWS

Threat Intelligence Platform

CyberNews is a full-stack cybersecurity intelligence platform developed as a weekly exercise in the Cybersecurity Fundamentals course. The project demonstrates practical application of web development, API integration, and AI-assisted analysis within a security operations context.

Hany Dhannoon

10.05.2026

CYBERNEWS

Threat Intelligence Platform

SECTION 1

EXECUTIVE SUMMARY

CyberNews is a full-stack cybersecurity intelligence platform developed as a weekly exercise in the Cybersecurity Fundamentals course. The project demonstrates practical application of web development, API integration, and AI-assisted analysis within a security operations context.

The platform aggregates live threat intelligence from multiple industry sources, presents it through a modern dark-themed dashboard, and provides AI-powered analysis using Google Gemini 2.5 Flash. It covers six core cybersecurity domains: threat intelligence aggregation, vulnerability management, indicator-of-compromise tracking, MITRE ATT&CK framework mapping, incident response workflow, and detection rule generation.

This report documents the project architecture, implementation details, strengths and limitations, and proposed future enhancements.

demonstrates the integration of modern web development practices with real cybersecurity workflows within the scope of a single weekly class exercise

9	5	5	AI
Sections / Modules	Live Data APIs	Detection Rule Formats	Gemini 2.5 Flash

SECTION 2

PROJECT OVERVIEW

Background & Objectives

This project was assigned as a weekly exercise to reinforce practical skills in:

- Building full-stack web applications with a Python/Flask backend
- Integrating real-world cybersecurity APIs and data feeds
- Implementing AI-powered analysis pipelines
- Designing professional, security-themed user interfaces
- Structuring modular, maintainable Python code

Technology Stack

Layer	Technology	Purpose
Backend	Python 3.12 + Flask 3.x	REST API server, route handling, data fetching
AI Engine	Google Gemini 2.5 Flash	Threat analysis, rule generation, IR playbooks
Frontend	HTML5 + CSS3 + Vanilla JS	Dashboard UI, real-time feed, interactive panels
Fonts	Orbitron, Rajdhani, Share Tech Mono	Cyberpunk / SOC aesthetic typography
Config	python-dotenv / .env file	Secure API key management
Data: CVEs	NVD / NIST API v2	Live CVE vulnerability feed
Data: News	HackerNews Algolia API	Cybersecurity news aggregation
Data: IOC	abuse.ch URLhaus	Live malicious URL indicators
Data: MITRE	Embedded ATT&CK dataset	Technique matrix and tactic mapping

SECTION 3

APPLICATION FOLDER & FILE STRUCTURE

The project follows a conventional Flask application layout with clean separation between backend logic, AI engine modules, frontend assets, and configuration files.

Project Directory Tree

```
cybernews/
|
+-- app.py                # Flask application & all REST routes
+-- ai_analysis_engine.py # AI module: threat analysis, CVE, IOC, MITRE, IR
+-- rule_generator.py     # AI module: detection rule generation & validation
+-- requirements.txt      # Python dependencies (flask)
+-- .env                  # API keys (GOOGLE_API_KEY, GEMINI_MODEL, etc.)
+-- .gitignore            # Excludes .env from version control
+-- README.md            # Setup and usage documentation
|
+-- templates/
|   +-- index.html        # Single-page application HTML
|
+-- static/
|   +-- css/
|       +-- style.css     # Dark cyberpunk theme (550+ lines)
|   +-- js/
|       +-- main.js       # All frontend logic, API calls, UI (800+ lines)
```

File Descriptions

app.py — Flask Application Server

The central entry point. Defines all Flask routes, imports functions from `ai_analysis_engine.py` and `rule_generator.py`, and handles real-data fetching from NVD, HackerNews, and abuse.ch. Exposes 14 REST API endpoints covering data feeds and AI analysis.

ai_analysis_engine.py — AI Analysis Engine

Standalone Python module responsible for all AI-powered threat analysis. Contains six public functions (*`analyse_threat`*, *`generate_ir_playbook`*, *`explain_mitre_technique`*, *`summarise_cve`*, *`classify_ioc`*, *`stream_analysis`*) that call the Gemini 2.5 Flash API via *`urllib`*. Includes its own *`.env`* loader, SSL context, and self-test suite.

rule_generator.py — Detection Rule Generator

Standalone Python module for AI-driven security rule generation. Supports five formats (Sigma, YARA, Suricata, Splunk SPL, KQL) and five functions (***generate_rule***, ***validate_rule***, ***explain_rule***, ***suggest_improvements***, ***batch_generate***). Includes Flask convenience handlers and a 7-test self-test suite.

templates/index.html — Frontend SPA

Single HTML file containing all nine dashboard sections: CTI Platform, SOC Dashboard, News Aggregation, IOC Tracking, AI Analysis, MITRE ATT&CK, Incident Response, Detection Rules, and Real-Time Monitoring.

static/css/style.css — Stylesheet

550+ line CSS file implementing the dark cyberpunk aesthetic with CSS custom properties, scanline overlay, grid background, glowing accents, animated elements, and full responsive layout.

static/js/main.js — Frontend Logic

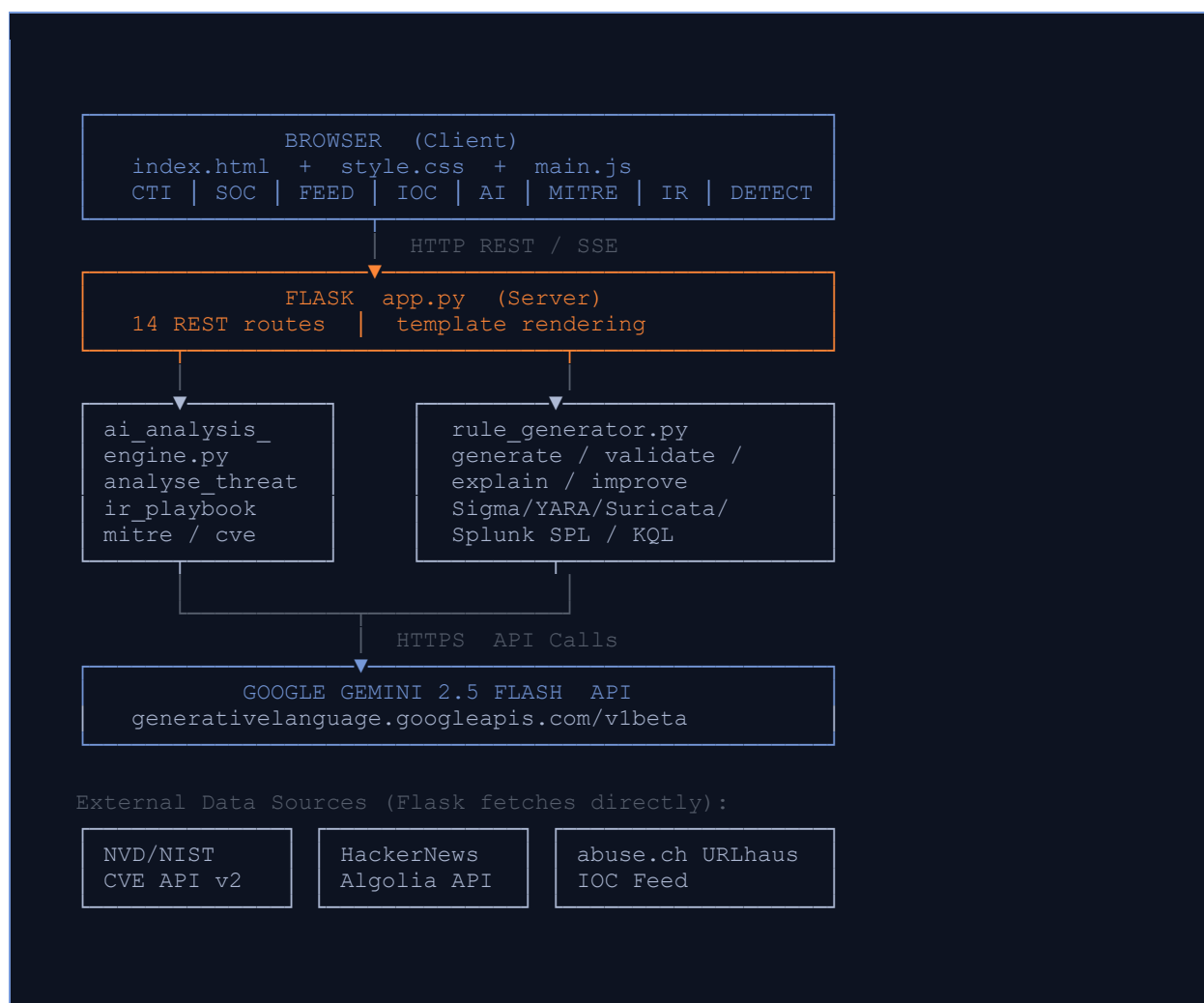
800+ line JavaScript file handling all API calls, DOM manipulation, real-time feed simulation, MITRE matrix rendering, Kanban drag-and-drop, syntax highlighting, and the AI typewriter effect.

SECTION 4

SYSTEM ARCHITECTURE

High-Level Architecture Diagram

The platform uses a client-server architecture. The browser communicates exclusively with the Flask backend via REST/SSE. The backend fans out to external APIs and the Gemini AI model.



API Endpoint Map

Method	Endpoint	Module	Description
GET	/api/cves	app.py	Live CVE feed — NVD NIST API v2

Method	Endpoint	Module	Description
GET	/api/news	app.py	Cybersecurity news — HackerNews
GET	/api/ioc	app.py	Malicious URLs — abuse.ch URLhaus
GET	/api/stats	app.py	Dashboard threat statistics
POST	/api/ai-analyze	ai_analysis_engine	General threat topic analysis
POST	/api/ai-cve	ai_analysis_engine	CVE analyst briefing
POST	/api/ai-ioc	ai_analysis_engine	IOC classification & attribution
POST	/api/ai-mitre	ai_analysis_engine	MITRE ATT&CK technique deep-dive
POST	/api/ai-playbook	ai_analysis_engine	NIST IR playbook generation
GET	/api/stream-analysis	ai_analysis_engine	SSE streaming threat analysis
POST	/api/generate-rule	rule_generator	Detection rule generation
POST	/api/validate-rule	rule_generator	Rule syntax validation
POST	/api/explain-rule	rule_generator	Plain-English rule explanation
POST	/api/improve-rule	rule_generator	Rule hardening suggestions

SECTION 5

FEATURE MODULES

The platform is divided into nine functional modules accessible from the top navigation bar.

#	Module	Data Source	Key Functionality
1	CTI Platform	NVD NIST API v2	CVE search by keyword/severity, CVSS scores, NVD links
2	SOC Dashboard	Simulated / internal	Threat level meter, attack vector bars, geo dot map, live alerts
3	News Aggregation	HackerNews Algolia	Searchable security news feed, quick-topic filters
4	IOC Tracking	abuse.ch URLhaus	Live malicious URLs, status, threat tags, date tracking
5	AI Analysis	Gemini 2.5 Flash	Typewriter threat analysis, quick-topic buttons, context input
6	MITRE ATT&CK	Embedded dataset	Interactive 10-tactic matrix, 40+ techniques, AI explain
7	Incident Response	Gemini 2.5 Flash	Kanban board (drag-and-drop), AI playbook generator
8	Detection Rules	Gemini 2.5 Flash	5 rule formats, syntax highlight, copy, validate, explain
9	RT Monitoring	Simulated stream	Live event feed, type filter, event counters, pause/clear

SECTION 6

PROS & CONS ANALYSIS

An honest assessment of the platform's current strengths and limitations as a class exercise prototype.

STRENGTHS	LIMITATIONS
✓ Real Live Data Pulls actual CVEs, news, and IOCs from production threat intel APIs rather than mocked data ✓ Zero Third-Party SDK Uses only Python stdlib (urllib, json, ssl) — no requests or anthropic package needed ✓ Modular Architecture ai_analysis_engine.py and rule_generator.py are self-contained, independently testable modules ✓ Multi-Format Rules Detection rules generated in 5 industry-standard formats covering all major SIEM/IDS platforms ✓ Secure Key Management API keys stored in .env, excluded from git, with .env.example for safe sharing ✓ Drag-and-Drop IR Board Functional Kanban incident board with drag-and-drop between status columns ✓ AI Explainability Every AI output explains its reasoning; rules include analyst notes ✓ Self-Test Suites Both AI modules include standalone self-test scripts (python ai_analysis_engine.py) ✓ Responsive Design Dashboard adapts to mobile/tablet with CSS grid and media queries ✓ MITRE ATT&CK Integration Interactive matrix with 10 tactics and 40+ techniques, AI-powered technique explanations	✗ No Authentication The platform has no login system — anyone with the URL can access all data and AI features ✗ No Data Persistence Incidents created on the Kanban board are lost on page refresh — no database backend ✗ AI API Dependency All AI features fail if the Gemini API key is missing or quota is exceeded ✗ Simulated Real-Time The RT monitoring feed and SOC alerts are simulated, not actual network events ✗ Rate Limits on NVD NVD API can throttle requests — no retry logic or caching implemented ✗ Single-Page Only No routing — refreshing navigates away from the current section ✗ No Rule Export Generated detection rules can be copied but not downloaded as files or pushed to SIEMs ✗ SSL Verification Off Some data fetches disable SSL cert verification for compatibility, reducing security ✗ No Input Sanitisation Limited server-side validation on AI prompt inputs ✗ Static MITRE Data ATT&CK data is hardcoded in JavaScript rather than fetched from the live MITRE STIX API

SECTION 7

FUTURE UPGRADE POSSIBILITIES

The following enhancements would elevate CyberNews from a class prototype to a production-capable threat intelligence platform.

Upgrade	Priority	Description
User Authentication	HIGH	JWT-based login with role separation (analyst / admin / read-only). Integration with LDAP/SSO for enterprise deployment.
PostgreSQL Backend	HIGH	Replace in-memory state with a relational database. Persist incidents, saved CVEs, custom IOCs, and generated rules across sessions.
Live MITRE STIX Feed	HIGH	Replace hardcoded ATT&CK data with dynamic fetching from MITRE's official STIX 2.1 API, enabling automatic updates.
SIEM Rule Export	MEDIUM	Push generated Sigma/KQL/Suricata rules directly to Splunk, Microsoft Sentinel, or Elastic SIEM via their APIs.
Threat Scoring Engine	MEDIUM	Assign risk scores to IOCs and CVEs based on CVSS, recency, exploitability, and AI-assessed severity context.
Real Network Monitoring	MEDIUM	Integrate with Zeek, Suricata, or OSSEC to display actual network events instead of simulated RT feed data.
Multi-Source IOC Feeds	MEDIUM	Add CIRCL, AlienVault OTX, Abuse.ch Feodo Tracker, and CISA KEV alongside URLhaus for broader indicator coverage.
Threat Actor Profiles	LOW	Dedicated pages for APT groups with linked MITRE techniques, historical campaigns, and AI-generated actor summaries.
Report Generation	LOW	Auto-generate PDF/DOCX incident reports and threat briefings from platform data with one click.
Container Deployment	LOW	Dockerise the application with docker-compose for reproducible, isolated classroom and production deployment.
Collaborative Incidents	LOW	Multi-analyst real-time collaboration on the IR board using WebSockets (Flask-SocketIO) for shared incident management.
Notification Alerts	LOW	Email/Slack/webhook notifications for critical CVEs, new CISA KEV entries, or high-severity IOC matches.

Suggested Implementation Roadmap

Phase	Timeline	Deliverables
Phase 1	Weeks 5–6	User authentication, PostgreSQL persistence, live MITRE STIX feed
Phase 2	Weeks 7–8	SIEM rule export, multi-source IOC feeds, threat scoring engine
Phase 3	Weeks 9–10	Real network monitoring integration, collaborative IR board (WebSockets)
Phase 4	Weeks 11–12	Container deployment (Docker), report generation, notification system

SECTION 8

LEARNING OUTCOMES

This weekly exercise provided hands-on exposure to the following competencies:

Web Development	Cybersecurity Concepts	AI / API Integration
• Flask routing and REST API design • Jinja2 templating • Vanilla JS async/await and fetch() • CSS Grid and custom properties • Responsive layout design	• CVE scoring (CVSS v2/v3) • IOC types and threat feeds • MITRE ATT&CK framework • Incident response phases (NIST) • Detection rule engineering	• LLM prompt engineering • REST API integration (urllib) • SSE streaming implementation • Environment-based key management • AI output parsing and rendering

SOME USEFUL NOTES

where is the response - incident board incident saved?

The incidents are saved in **browser localStorage** — not on disk, not in a database, not in any file on your computer.

What is localStorage?

It's a small key-value store built into every web browser. Each website gets its own isolated storage area. The data lives **inside the browser's internal profile folder**, completely separate from project files.

Where to find it on Windows?

The browser holds it — not a readable file you can open in Notepad. But here's where each browser physically stores its profile data:

Chrome / Edge

C:\Users\<YourName>\AppData\Local\Google\Chrome\User Data\Default\Local Storage\leveldb\

C:\Users\<YourName>\AppData\Local\Microsoft\Edge\User Data\Default\Local Storage\leveldb\

Firefox

C:\Users\<YourName>\AppData\Roaming\Mozilla\Firefox\Profiles\<random>.default\webappsstore.sqlite

These are binary/database files — you can't read them directly. The browser manages them internally.

*Where to find it on Linux***Chrome / Chromium**

~/.config/google-chrome/Default/Local Storage/leveldb/

~/.config/chromium/Default/Local Storage/leveldb/

Firefox

~/.mozilla/firefox/<random>.default/webappsstore.sqlite

The easy way to see your saved incidents

Open your browser's **DevTools** while on the CyberNews page:

F12 → **Application tab** → **Local Storage** → <http://localhost:5000>

You'll see the key:

cybernews_incidents_v1

with the full JSON value of all your saved incidents.

Important limitations to understand

Property	Value
Storage key	cybernews_incidents_v1
Scope	Only accessible from http://localhost:5000 in that specific browser
Capacity	~5 MB per origin
Persistence	Survives browser close/reopen ✓
Shared across browsers?	✗ No — Chrome and Firefox have separate storage
Shared across computers?	✗ No
Cleared when?	If you clear browser history/cache, or call <code>irClearStorage()</code>
Accessible by Flask/Python?	✗ No — server cannot read it

SECTION 9

CONCLUSION

CyberNews v3.0 successfully demonstrates the integration of modern web development practices with real cybersecurity workflows within the scope of a single weekly class exercise. The project goes significantly beyond a basic prototype by incorporating live threat intelligence feeds, AI-powered analysis across six specialised domains, and a fully functional detection rule engineering pipeline.

The modular Python architecture — with `ai_analysis_engine.py` and `rule_generator.py` as independent, testable units — reflects professional software engineering practices and provides a solid foundation for future enhancement. The use of Google Gemini 2.5 Flash as the AI backend demonstrates practical LLM integration without reliance on heavyweight SDKs, using only Python's standard library for API communication.

While the platform has clear limitations appropriate to its prototype status — notably the absence of authentication, persistent storage, and real network data — these are well understood and mapped to a concrete upgrade roadmap. The identified limitations are not architectural flaws but deliberate scope decisions for a time-bounded class exercise.

From a learning perspective, this project provided comprehensive exposure to the full stack of a modern security operations tool: from data ingestion and AI analysis to frontend visualisation and incident management workflow. The skills exercised — Flask API design, prompt engineering, threat intelligence feeds, MITRE ATT&CK, NIST IR frameworks, and detection engineering — directly map to entry-level SOC analyst and security engineer roles.

